

Java Threads

Multitasking and Parallel Execution

CODEHIVE

CODEHIVE

1. What are Threads?

Threads allow a program to operate more efficiently by performing multiple tasks simultaneously. Instead of waiting for one task to finish before starting the next, threads enable **Parallelism**.

Common Uses:

- Running background data syncs.
- Maintaining a responsive UI while processing files.
- Handling multiple client requests in a server.

2. Creating Threads

There are two primary paradigms for creating a thread in Java. Each has specific architectural advantages.

Method 1: Extending Thread

Direct inheritance from the Thread class. Simple, but "uses up" your single inheritance slot.

```
public class Worker extends Thread {  
    public void run() {  
        System.out.println("Thread logic here");  
    }  
}
```

3. The Runnable Interface

Implementing **Runnable** is the industry preference because it allows your class to extend another class while still being capable of running as a thread.

```
public class Main implements Runnable {  
    public void run() {  
        System.out.println("Running via Runnable...");  
    }  
}
```

4. Launching the Thread

To start a thread, you must call the `start()` method. ****Crucial:**** Never call `run()` directly, as it will execute in the current thread rather than a new one.

```
Main obj = new Main();  
Thread thread = new Thread(obj); // Pass Runnable to Thread constructor  
thread.start(); // This creates the new execution context
```

5. Hierarchy vs Interface

Feature	Extends Thread	Implements Runnable
Inheritance	Locked to Thread	Flexible (can extend others)
Usage	Simpler for beginner tasks	Standard for production apps

6. Concurrency Risks

Because threads run side-by-side, the order in which they access variables is **unpredictable**. This creates race conditions.

Problem Example: If two threads increment a shared integer `x++` at the exact same millisecond, they might both read '5' and both write '6', losing one increment entirely.

7. Shared Memory Hazard

Look at how the main program and a thread interact with the amount variable without synchronization.

```
public static int amount = 0;

public static void main(String[] args) {
    Main thread = new Main();
    thread.start();
    // This value is unreliable!
    System.out.println(amount);
}
```


8. Using `isAlive()`

A simple (but brute-force) way to solve concurrency is to wait until a thread finishes its work using the `isAlive()` method.

```
thread.start();

// Block main while thread is working
while(thread.isAlive()) {
    System.out.println("Synchronizing...");
}

// Now amount is safe to read
System.out.println("Final: " + amount);
```

9. Summary & Final Tips

- **Multitasking:** Threads enable background processing.
- **Runnable:** Best for professional, decoupled code.
- **Start vs Run:** Always call `.start()`.
- **Safety:** Use `isAlive()` or `join()` to manage shared data states.

© 2026 CodeHive Academy | Advanced Java Series